FL-APU: A Software Architecture to Ease Practical Implementation of Cross-Silo Federated Learning

Fabian Stricker
Institute of Data-Centric Software Systems
Karlsruhe University of Applied Sciences
Karlsruhe, Germany
fabian.stricker@h-ka.de

José Antonio Peregrina
Institute of Data-Centric Software Systems
Karlsruhe University of Applied Sciences
Karlsruhe, Germany
jose_antonio.peregrina_perez@h-ka.de

David Bermbach
Scalable Software Systems Research Group
Technische Universität Berlin
Berlin, Germany
db@3s.tu-berlin.de

Christian Zirpins
Institute of Data-Centric Software Systems
Karlsruhe University of Applied Sciences
Karlsruhe, Germany
christian.zirpins@h-ka.de

2024 2nd International Conference on Federated Learning Technologies and Applications (FLTA) | 979-8-3503-5481-2/24/$31.00 ©2024 IEEE | DOI: 10.1109/FLTA63145.2024.10839980

***Abstract*—Federated Learning (FL) is an upcoming technology that is increasingly applied in real-world applications. Early applications focused on cross-device scenarios, where many participants with limited resources train machine learning (ML) models together, e.g., in the case of Google's GBoard. Contrarily, cross-silo scenarios have only few participants but with many resources, e.g., in the healthcare domain. Despite such early efforts, FL is still rarely used in practice and best practices are, hence, missing. For new applications, in our case inter-organizational cross-silo applications, overcoming this lack of role models is a significant challenge.**

In order to ease the use of FL in real-world cross-silo applications, we here propose a scenario-based architecture for the practical use of FL in the context of multiple companies collaborating to improve the quality of their ML models. The architecture emphasizes the collaboration between the participants and the FL server and extends basic interactions with domain-specific features. First, it combines governance with authentication, creating an environment where only trusted participants can join. Second, it offers traceability of governance decisions and tracking of training processes, which are also crucial in a production environment. Beyond presenting the architectural design, we analyze requirements for the real-world use of FL and evaluate the architecture with a scenario-based analysis method.

***Index Terms*—Federated Learning, Inter-Organizational Collaboration, Software Architecture, Data Governance**

I. Introduction

State-of-the-art machine learning (ML) models require a large amount of data to achieve good results. Sometimes such data, however, is not available locally and can also not be obtained from third parties, e.g., due to data privacy issues or sensitive business secrets [1]. In such scenarios, federated learning (FL) [2] can help by training local models on sensitive data and only sharing the trained models which are then aggregated into a single global model. As a bleeding-edge technology, FL still faces challenges, e.g., regarding security [3], privacy [4], or quality [5]. Despite this, FL is already being used in practice for a few application domains, e.g., in Google's GBoard for improving mobile device word completion [6]. For inter-organizational cross-silo applications, however, public examples are still missing.

One such application is studied in the FederatedForecasts research project[1] which aims to improve short-term energy forecasting for wind and solar power generation. For this, it uses FL in a cross-silo scenario with multiple energy providers and a trusted third party as coordinator. While applying FL methods in FederatedForecasts, we identified several problems: First, while there is much research on FL fundamentals, e.g., [5], [7], [8], there is a lack of research on implementing real-world FL systems and maintaining them for an extended period of time. Second, while there are reference architectures for FL, such as [9], they focus on generic functionality that can be adapted to many different FL settings. Thus, they ignore domain-specific details and components that are relevant in the context of inter-organizational cross-silo applications.

In this paper, we add to closing the above mentioned gap through FL-APU, a scenario-based FL architecture focusing on collaboration. Following the FederatedForecasts scenario, we identify real-world requirements and tailor existing reference architectures for practical industry application. In this regard, we make the following contributions:

- We identify and define requirements for cross-organizational FL (Section III).
- We define and discuss roles which are necessary for inter-organizational FL architectures (Section IV).
- We describe two reference architectures for FL servers (Sections V) and clients (Section VI).

This work was co-funded by the German Federal Ministry of Education and Research (BMBF) under Grant 13FH587KX1 (FederatedForecasts) and the European Union in the Interreg Upper Rhine programme (project Aura.ai).

[1] https://www.h-ka.de/en/idss/projects/federatedforecasts

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 07,2026 at 19:05:03 UTC from IEEE Xplore. Restrictions apply.

- We identify and analyze key features such as governance and metadata management (Section VII).
- We validate our design through a scenario-based evaluation of the architecture (Section VIII).

II. Related Work

This section discusses related work on FL systems architecture and development methods. Bonawitz et al. [10] created a system design for a cross-device scenario with billions of devices. Handling such numbers of devices leads to several problems, e.g., client connectivity, resource limitations, and scalability in regard to the aggregation and the coordinators that manage different populations of participants. In addition, Stojkovic et al. [11] also proposed an architecture for a similar problem. While scalability is necessary, they mention additional challenges, such as slow release cycles and low client participation. These issues are less pronounced in cross-silo, cross-organizational scenarios because only a few participants with many resources usually participate in the training. In addition, critical organizational aspects such as metadata management and a platform for collaboration are missing. Cheng and Long [12] propose Federated Learning Operations (FLOps), a methodology to develop FL cross-silo systems. In comparison, they provide a methodological perspective and do not focus on architecture design. Lo et al. [9] proposed a pattern-oriented reference architecture for general FL. Their architecture includes fundamental components (e.g., model trainer, aggregator, evaluator, monitoring, and job creator) that provide a good basis for creating customized FL processes. However, we focus on the practical use of FL in cross-silo scenarios, including cooperation and collaboration of multiple companies (e.g., negotiation of the FL process configuration). Here, some aspects are missing due to the abstraction level and collection of patterns used, e.g., metadata management, a platform for collaboration, data validation, and reporting. There are a few companies that already offer FL frameworks, such as IBM [13], NVIDIA [14], and Flower Labs [15]. However, frameworks are primarily used as a foundation to ease implementation for building experimental or more extensive applications upon it. Hence, a simple framework without management, monitoring, collaboration, or security can not be used as a production-ready application. An example of building upon a framework is the work of Galenda et al. [16]. They propose an FL platform that builds upon the Flower Framework [15] and extends it with a graphical user interface. The goal is to reduce the technical knowledge required to deploy FL in a production-ready environment. However, even with the extension, the platform only provides basic functionality and misses components for collaboration, model deployment, and monitoring. In the case of software, Galtier and Marini [17] proposed Substra, an open-source FL software. Substra uses a decentralized FL architecture focusing on collaboration, traceability, and privacy. It consists of different components, such as an asset network to exchange algorithms and models, but it also uses ledgers to trace the training and results. Compared to our approach, the collaboration is only focused on data and is missing a platform to negotiate the configuration of the FL process. Furthermore, we focus on centralized FL. Therefore, there is a difference in the communication and components of the architecture.

III. Federated Learning in Practice

In the context of FederatedForecasts, we have analyzed the cross-organizational aspects that an FL implementation needs to consider in practice: First, it needs to preserve privacy to prevent the leakage of valuable information to other companies or third parties. Second, it has to ensure a fair process where all participants profit and are rewarded for the value of their contributions. Since all participants are competitors, participating in an unfair process could lead to disadvantages in the market. Third, to break down the result of the FL process and create a transparent process, the decisions and information of the process have to be tracked. Fourth, the context of the project is energy production, which is considered critical infrastructure. Hence, the quality and robustness of the predictions are essential factors, as well as the reliability of the process.

Beyond the project context, the companies have additional use cases and requirements for the architecture design. During the prototyping phase, we identified several problems with the communication between company infrastructures. First, each of them is subject to different security policies. Second, individual experiences with different communication technologies influence the perception of their suitability. Accordingly, we systematically analyzed these areas in individual meetings with the security officers from each involved company, where we discussed the requirements regarding communication of information, security, and privacy. Besides the security-related concerns, we also discussed how the companies can include their requirement and experience in the FL process to achieve a benefit for all participants. The discussions resulted in the following essential requirements:

1) For the communication between FL Server and FL Client, a virtual private network (VPN) solution is ruled out because of potentially malicious network-level access.
2) Using a remote procedure call (RPC) that does not ensure HTTP2-based communication over the internet is likewise undesired to prevent remote execution of malicious code.
3) The trained models should be stored and tracked because historic models from earlier training runs could achieve better performance.
4) A decision-making process is essential so that each company can include its experience with ML models in the training process.
5) Participating companies require sovereign management of their clients to increase data privacy in the presence of involved market competitors.
6) An external server is not allowed to send messages that start operations within the company infrastructure.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 07,2026 at 19:05:03 UTC from IEEE Xplore. Restrictions apply.

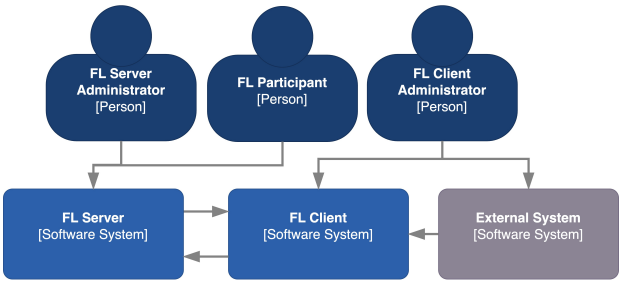


Fig. 1. FL-APU: Overview of software systems and their interactions with their surroundings

IV. Architectural Overview and Role Model

In this and the following two sections, we introduce FL-APU, our proposal for an architecture designed for practical use of FL in real-world applications with inter-organizational and cross-silo characteristics. FL-APU builds on the pattern-oriented reference architecture for FL (FLRA) [9] and refines the architectural design for cross-silo cross-organizational scenarios. Furthermore, we extend the base architecture with new components required for the cooperation of organizational participants (i.e., companies) and the integration of the trained model into external applications. From a practical perspective, the operation of a distributed FL system can be broken down into task areas related to its usage, monitoring, and maintenance. While monitoring can be done automatically to some extent, other task areas require the active involvement of different types of staff. In order to structure and distribute their responsibilities, role definitions can be defined. In particular, roles related to systems management are important in widely distributed settings such as FL systems. Figure 1 shows the system context of our proposed architecture, including the different roles that interact with the system.

Our proposed architecture comprises two software systems, the FL Server and the FL Client. Both components interact with each other to coordinate and train an ML model. Besides the systems, different roles are involved in the task areas of operating the system:

- **FL Server Administrator**: Manages the FL Server and monitors the overall FL process, e.g., can start test runs to validate system functionality.
- **FL Participant**: Participates in the decision process for the FL training and process parameters, e.g., deciding on a data format or hyperparameters.
- **FL Client Administrator**: Manages the FL Client and monitors the process, e.g., defines the threshold for deploying or replacing models.

Besides the roles interacting with the system, we also consider an external system interacting with the FL Client. The goal is to access the deployed ML model as a service for inference and to use the predictions in other products.

V. FL Server Architecture

In this section, we propose the server architecture of FL-APU. The container view of the FL Server is shown in Figure 2. Containers are deployable units that consist of multiple components and focus on fulfilling a specific task in the system. Due to the size of the architecture, a visualization on component-level is omitted for the sake of brevity. We describe the major parts in the following.

Governance and Management Website The Governance and Management Website enables the FL Server Administrator to access the system and manage the server. For example, the administrator can force the deployment of a specific model to a client or perform test runs by manually creating an FL Job to run specific configurations. In addition to managing the server, the FL Server Administrator is responsible for registering the FL participants in the Client Management. This is needed to ensure that only trusted participants can join the FL process. Furthermore, the FL Server Administrator and participants can view the current state of the FL process. In addition, the FL Participant can take part in the governance negotiation process, which is needed to define the data structure in the data silos, as well as the hyperparameters such as the FL training rounds. For example, in our use case, the resolution of the time series data has to be defined.

Governance Manager The Governance Manager consists of two components, a *Governance Cockpit* and an *application programming interface* (API). The Governance Cockpit manages the negotiation process and stores all decisions in a governance contract. The contract can be sent to the Job Creator to define an FL Job, including hyperparameters and requirements. This is required to allow the participants to include their goals in the process and to collaborate effectively. We explain the governance aspects of the architecture in Section VII.

Client Management The Client Management consists of three components: *User Management*, *Client Registration*, and *Client Registry*. The first one is needed to register the FL participants with a user account and perform authentication of clients. The next one is the Client Registration, which accepts registration requests and validates them before they are added to the Client Registry. Hence, only legitimate clients can participate in an FL process. This container is important for increasing the security of the system, which is essential for the industry scenario.

Job Creator This container is responsible for creating an FL Job from a governance contract or input from the FL Server Administrator. An FL Job contains all parameters required for an FL process, including the training rounds, the train-test-split ratio, evaluation metrics, and more. The resulting FL Job is sent to the FL Manager.

FL Manager The goal of the FL Manager is to handle the whole FL process. It consists of multiple components, including an *FL Run Manager* that is responsible for managing the other components and starting the process once all required clients are connected to the Client Management. In addition, the FL Run Manager can repeat the FL process with different hyperparameters if the participants decide to use hyperparameter optimization. The managed components are the *Data Validator*, the *Preprocessing Coordinator*,

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 07,2026 at 19:05:03 UTC from IEEE Xplore. Restrictions apply.

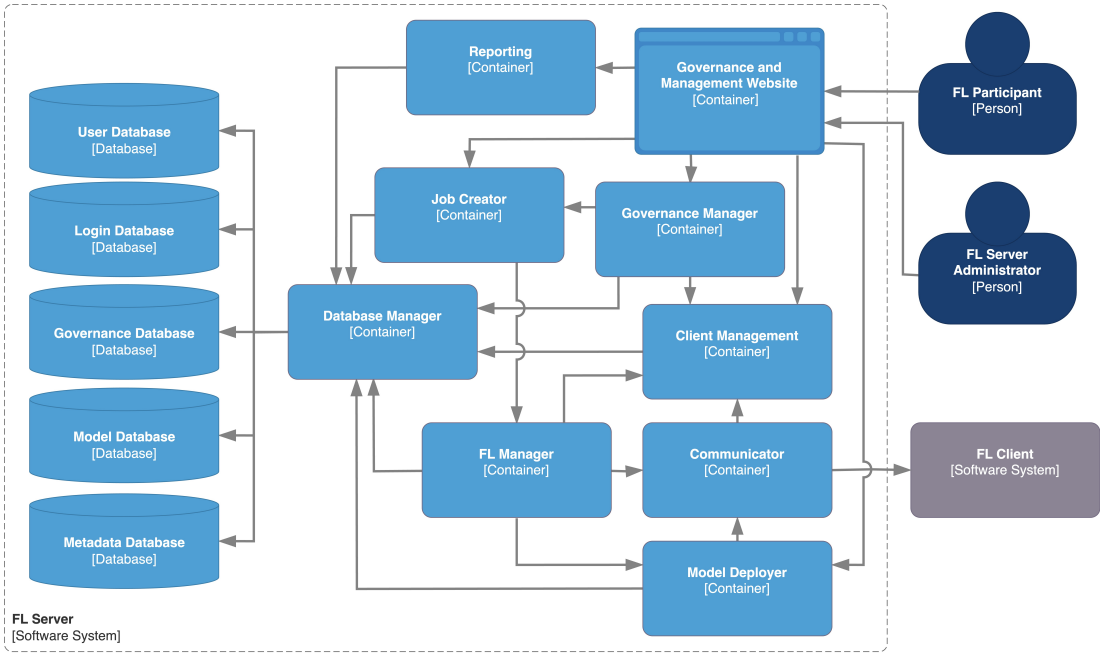


Fig. 2. FL-APU: Server architecture as deployable containers; arrows are unidirectional container relationships; containers consist of multiple components

the *Training Coordinator*, the *Model Aggregator*, and the *Evaluation Coordinator*. The first one is responsible for validating that the data format decisions are upheld by validating the data structure on the client's side before the training process starts. This is important for increasing the robustness of the process by detecting wrong data structures. The Preprocessing Coordinator informs the client on how to preprocess the data. Next, the Training Coordinator tells the client how to configure the training process. Similarly, the Evaluation Coordinator informs the client how to perform the evaluation. Furthermore, it is also responsible for measuring the client contribution. In our scenario, it is important to ensure that each participant contributes equally and that they are compensated based on the value of their contributions. The last component is the Model Aggregator, which aggregates client models into the global model. Generally, the Coordinator components and the Data Validator have a counterpart on the client side.

Communicator The Communicator manages all the communication between the FL Server and the FL Clients. It consists of a *Communication Manager* who handles communication with each client and handles encryption and compression of messages. Each client has a corresponding Communicator container to receive information from the FL Server.

Model Deployer Once the training has been completed, the FL Run Manager triggers the Model Deployer to deploy the latest global model on the clients. Furthermore, the FL Administrator can deploy a specific model on the clients if an FL Participant requests it.

Database Manager The Database Manager receives all information regarding users, login, governance, trained models, and metadata. This information is stored in the corresponding databases to track the trained model and the overall process.

Reporting The Reporting component reads the stored information and displays it in a detailed report on the website. This is important for preparing a suitable visualization for the FL Participant.

VI. FL Client Architecture

Since FL commonly consists of a server and clients, we introduce the architecture for the FL Client as shown in Figure 3. Again, we show the container view of the FL Client and introduce the main containers in the following.

Management Website The Management Website allows the FL Client Administrator to manage the FL Client. This includes defining the thresholds for model deployment, defining a model personalization process, monitoring the deployed model, and monitoring the overall FL Client. The idea is to enable participating companies to control their FL Clients because these are parts of their individual enterprise IT landscapes. Besides the website, this container also includes a *Model Subscription API* component for external systems to perform predictions and use the results in the application. Hence, it enables the integration of the FL Client into an external software system.

FL Client Model Deployer This container consists of five components. The first one is the *FL Client Manager*, who handles and tracks the deployment process and prepares the information for the Management Website. The next component is *Model Personalization*, which is responsible for personalizing the global model received from the FL Client Manager. Another component is the *Decision Maker*,

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 07,2026 at 19:05:03 UTC from IEEE Xplore. Restrictions apply.

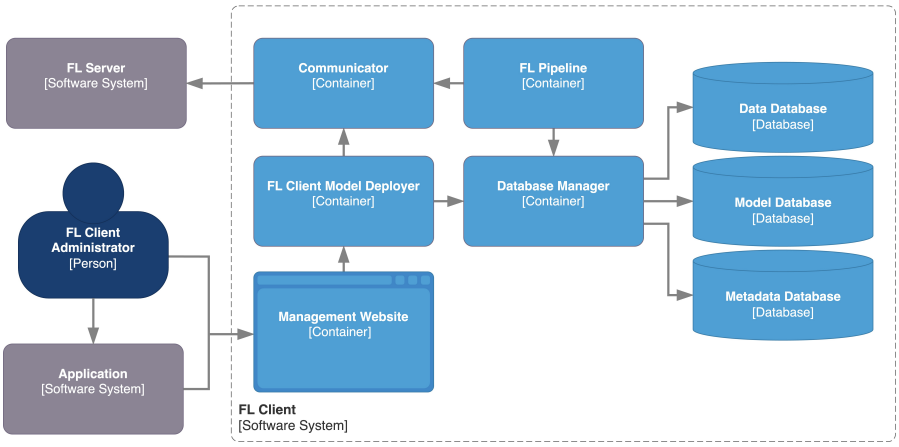


Fig. 3. FL-APU: Client architecture as deployable building blocks; Arrows are unidirectional and represent relationships between the containers; containers consist of multiple components

which receives the personalized model to validate whether it fulfills the requirements for deployment. If the model fulfills the requirements, it is deployed to the next component, the *Inference Manager*. The Inference Manager handles the request from the Model Subscription API and performs the inference on the trained model. The last component is *Model Monitoring*, which uses a fixed test dataset to evaluate the performance of the deployed model. Once the model reaches a defined threshold or a model is not deployed, the FL Client Manager notifies the FL Client Administrator. The administrator can inform the FL Participant to request a different version of the global model or a new training process. The validation and monitoring of the deployed model is crucial for a real-world scenario because undetected changes in the model performance can lead to fatal risks.

FL Pipeline The FL Pipeline consists of one component each for *Data Validation*, *Data Preprocessing*, *Model Trainer*, and *Model Evaluator*. These are the coordinator's counterparts on the FL Server. The Data Validation executes the configuration to check whether the data structure is valid. The next component, Data Preprocessing, takes parameters to execute preprocessing operations. The Model Trainer trains the local model using private data and the configuration while the Model Evaluator component uses private test data to validate the model performance of the global model.

Communicator The Communicator is similar to the one in the FL Server but without managing all client connections.

Database Manager The Database Manager is also similar to the one in the FL Server. In comparison, the client must store the training and evaluation data, client models, and metadata.

VII. Features for Practical FL

In FL-APU, we have different containers to satisfy specific cross-organizational requirements (see Section III).

In this section, we explain the features of the architecture that aim to tackle these requirements. While other features are vital, e.g., the model training, we only focus on the features rarely discussed in academia which, however, are essential for the adoption of the architecture in practice.

Governance In a default setting without governance aspects, the coordinator of the FL system decides how the FL process is conducted. In this case, the participants only decide if they want to participate. For example, the goal of Google's Gboard is to improve the application, and achieve better quality for the clients [6]. In this case, the one that benefits most is Google because they can offer a better product and may attract more users. The participants (i.e., the users) cannot influence the process. However, in a cross-silo use case, the participating companies expect to profit from the resulting model. At the same time, the FL Server is an essential component that enables cooperation between the participants and creates the global model, but mostly does not benefit from the FL process. In this regard, the participants must be able to negotiate the FL process configuration to include their input. For example, the experience with different model architectures or the requirement of explainable models are possible inputs. Furthermore, the companies will only participate if they receive a benefit. The goal of Data Governance in FL [18] is to provide mechanisms that allow participants to negotiate the training parameters (e.g., the dataset for training and type of model). In order to integrate Data Governance into the architecture, two components are required. The first component is the Data Governance Cockpit, which contains all the governance mechanisms. In detail, it provides participants with mechanisms to define, configure, and agree on the training's goals, parameters, and restrictions. The managed information to negotiation processes within the scope of the Data Governance lifecycle [18]. From this agreement, a configuration is created and sent to the FL Manager, which starts the training. All operations performed within the Cockpit are recorded as provenance metadata. The second component is a web interface that provides a GUI for the user to interact with other participants in the decisions.

Metadata Management The term metadata refers to any data providing information about data. Within ML, metadata

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 07,2026 at 19:05:03 UTC from IEEE Xplore. Restrictions apply.

is used to track ML training experiments and the provenance of the different artifacts involved in the training. Hence, it is possible to compare the results achieved by different training runs and the changes that led to either an improvement or deterioration of the model performance. As the model performance can depend on many small details, like training data characteristics or hyperparameter tuning, it is necessary to compare the evolution of changes within these details to the model's performance. In FL, tracking and comparing these changes is even more important because the performance evaluation is performed by multiple parties, each possessing different data. Following the work in [19], this proposal defines a component where metadata are managed. Provenance metadata allows participants to analyze who performed the operations and the corresponding outcome of the operations. Experiment tracking metadata provides information on the training results and configuration without ever sharing training data or information on the contents of such data. The goal is to prevent the execution of any operation or action that would collide with the concept of FL, where privacy is preserved by design.

Data Validation A good-performing ML model in FL needs to consider multiple aspects, such as the problem of data heterogeneity, a suitable aggregation method, and model personalization [5], [20]. In addition, it is vital to have identically structured data on the clients because it is a requirement for horizontal FL [7], and homogeneous model architecture requires the same input size. In our architecture, the properties of the data are negotiated during the governance process to solve this issue. For a more robust FL process, we need to validate that all FL Clients use the correct data structure and that the values are within valid ranges. For example, the frequency in a time series dataset should be the same for all FL Clients. If the data validation fails on a client, the FL Run Manager will identify the client through the Client Management and pause the process. The information is stored and reported on the website to inform the FL Participant. A similar process is performed on the FL Client to notify the FL Client Administrator.

User Authentication In FL, preventing unknown clients from joining the training process is essential because such a client could try to steal the global model [21] or disturb the overall training process with poisoned data [3], [22]. In a cross-silo scenario, however, there is only a small number of clients. Hence, detecting a single malicious client is more realistic than in a cross-device scenario. Despite this, we still have to ensure that only the participating companies can join the process. The Client Management container mitigates the problem of unregistered clients by combining the user participating in the negotiation process with the device performing the client-side training. Such a process could look like this in practice:

1) The participating companies sign a contract with the FL service provider and receive login information for the governance website.
2) After the governance contract has been completed, each client receives an authentication token for their participating devices.
3) The participating device is using the token for authentication during the message exchange.
4) The FL Server is using Client Management to validate the tokens in the request.

With this process, the FL Participant has a separate authentication for the governance website and the participating device. Furthermore, the token changes after every FL training process. Hence, if the token is stolen, the FL Participant can report that their client did not receive a model, and a new process can be started with new tokens. If the same token is received from two different devices, then the FL Participant could add further information that enables a precise differentiation. However, the important aspect is that the FL Participant manually takes part in the governance negotiation. Consequently, this person can validate that they are still participating in the process and the client is theirs. The biggest issue of the process is that once the login information for the governance website is stolen, the company's identity is also stolen. Thus, a malicious FL Participant can participate in the negotiation process. In order to mitigate this problem, the login information should be kept safe, and state-of-the-art security methods (e.g., 2fa and continuous authentication) should be implemented. In case this happens, the low number of participants would allow the participants to easily restart the entire authentication process, starting from step 2 in the process above. In addition, we need to consider that multiple FL participants cooperate in the negotiation process. Hence, the impact of a single participant wanting to disturb the process is low. Furthermore, the configuration has to make sense; otherwise, the participants and the service provider will get suspicious. Finally, authentication is only one part of security, and additional mechanisms are required for more robust security guarantees.

Server Authentication and Privacy So far, we have only discussed user authentication, but server authentication is also necessary. Clients connecting to a server must ensure they are dealing with a legitimate server instead of a malicious third party. A malicious server could try to replace the original one in order to steal client information. This could be the model updates to perform inference attacks and retrieve private information [4]. Another possibility is that the malicious server could send a manipulated global model. Such a model could achieve good performance but introduce a backdoor to the clients [23]. There are state-of-the-art solutions for server authentication that can be implemented in the communicator to validate the genuineness of the server (e.g., certificates). Furthermore, this is essentially a consensus problem with byzantine failures for which various approaches such as [24] exist.

Regarding privacy, the clients should not trust the server even though a contract binds all parties in our scenario. Hence, common FL privacy mechanisms such as homomorphic encryption [25] are used in the architecture to increase privacy against leakage of private information from model updates.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 07,2026 at 19:05:03 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COLLECTIVE TASKS FOR SAAM

ID	Actor	Task
1	FL Participant	Participate in the negotiation
2	FL Participant	View FL Run history
3	FL Participant	Request new negotiation process
4	FL Participant	Request deployment of model
5	FL Server Admin	Create user accounts
6	FL Server Admin	Control the FL process
7	FL Server Admin	Create an FL Job
8	FL Server Admin	Set up a negotiation process
9	FL Client Admin	Set monitoring threshold
10	FL Client Admin	Set deployment threshold
11	FL Client Admin	Monitor the system
12	FL Client Admin	Manage model endpoint
13	FL Server	Prepare a report
14	FL Server	Create a FL Job from Information
15	FL Server	Turn governance result to FL Job
16	FL Server	Store/Retrieve information
17	FL Server	Run FL process
18	FL Server	Deploy a specific model
19	FL Server	Send messages to client
20	FL Server	Encrypt/Compress messages
21	FL Server	Authenticate client
22	FL Server	Generate device token
23	FL Server	Register client
24	FL Server	Monitor FL process
25	FL Server	Check registered clients
26	FL Client	Send messages to server
27	FL Client	Run FL Pipeline
28	FL Client	Store/Retrieve information
29	FL Client	Monitor local FL process
30	FL Client	Configure monitoring
31	FL Client	Configure personalization
32	FL Client	Configure model deployment
33	FL Client	Monitor deployed model
34	FL Client	Encrypt/Compress messages
35	FL Client	Perform model inference
36	FL Client	Perform model personalization
37	FL Client	Decide on model deployment
38	FL Client	Prepare report
39	FL Client	Trigger administrator notification
40	External Application	Send inference request

However, ensuring privacy is still an open challenge in FL.

VIII. SCENARIO-BASED EVALUATION

To evaluate the architecture design, we follow the scenario-based architecture analysis method (SAAM) [26]. This method verifies the architecture's design by identifying task scenarios and validating that the architecture can perform these scenarios with the current design. In the following, we refer to task scenarios as tasks. If the architecture does not fulfill a task, the method considers the changes needed to fulfill it and the resulting cost of the changes. The evaluation method consists of five steps. In the first step, we describe the architecture, including the containers and their relationships (see Section V and Section VI).

In the second step, we define tasks that the architecture must fulfill. In addition to the requirements in Section III and the system context in Section IV, we can deduce the necessary capabilities of an appropriate FL architecture in Table I.

Tasks 17 and 27 summarize the typical FL process steps, such as data validation, preprocessing, model training, aggregation of the client models, and evaluation.

TABLE II
LIST OF CONTAINERS AND THEIR ASSOCIATED TASKS

Container	Tasks
Reporting	2,13
Governance and Management Website	1-8
Job Creator	7,14,15
Governance Manager	3,15
Client Management	5,21,22,25
Database Manager	16
FL Manager	17,24,25
Communicator	19,20,21,23
Model Deployer	18
FL Pipeline	27
Management Website	9,10,11,12,39,40
Database Manager	28
FL Client Model Deployer	9-12,29-33,35-39
Communicator	26,34

In the third step, we categorize the tasks into direct and indirect ones. Direct tasks are supported by the architecture directly, and indirect tasks require changes to the architecture before they can be supported. In the case of the direct tasks, we analyze Table II. It shows the containers and the tasks they fulfill. Matching the fulfilled tasks with Table I, we can conclude that tasks 1 to 40 are direct tasks that the architecture can execute directly. Regarding indirect tasks, the changes in the future are mostly related to different aggregation algorithms, client contribution methods, evaluation methods, or metrics. However, the required containers have already been designed to adjust to different methods because these parts can be changed depending on the result of the negotiation and do not require a change in the architecture.

The requirements of the companies in Section III guided the overall design of the architecture. While the first two requirements focus on the actual implementation of the communication, the following three requirements influence the overall design of the process, such as the introduction of a governance component, the tracking of the training runs as well as the management interfaces on the FL Server and the FL Client. The last requirement influences the communication between the client and the server. While coordination and sending messages are vital for the server, the requirement prohibits direct execution of operations. Instead of direct communication, where the clients react to messages from the server, the clients have to be proactive in fetching configuration, model weights, and status updates from the server. In FL-APU, this affects the design of the Communicator. For example, a simple approach could be the implementation of a REST API to store information as resources. The clients periodically retrieve the resources and post client information as a new resource.

In the fourth step, we analyze the interactions with the containers. This step is usually done only for the indirect tasks and their changes. Since we do not have indirect tasks, we analyze the direct tasks to validate the separation of concerns and which containers fulfill which task. The containers and their tasks are shown in Table II.

As a result, several containers execute many tasks, such

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 07,2026 at 19:05:03 UTC from IEEE Xplore. Restrictions apply.

as the Management Website, FL Client Model Deployer, and the Governance Website. However, the containers have a higher abstraction level than the components. These containers include components that further distribute these tasks. In conclusion, the tasks are sufficiently split on the container level to fulfill the separation of concerns.

In the fifth step, we rank the importance of all tasks. While most tasks provide essential functionality to the overall FL Process, scenarios 7 and 14 are the least important because they focus on testing a specific FL Run. All the other tasks are essential and must be implemented to use the architecture. For example, tracking the process and generating reports without storing information such as metadata is complex. Another example is the encryption of messages; without it, the communication is insecure and would be rejected by the companies.

As a conclusion of SAAM, the architecture design can fulfill the required tasks; therefore, the design suffices for the overall use case at hand. We must, however, consider that SAAM only evaluates the architecture design in the context of the predicted usage and cannot anticipate usage scenarios that deviate from the intended path.

IX. Conclusion

Due to the lack of research on implementing and maintaining real-world FL systems for an extended period, realizing an architecture is challenging. In addition, existing reference architectures primarily focus on generic functionality. Thus, they ignore domain-specific details and relevant components in the context of inter-organizational cross-silo applications.

We proposed an architecture design for the real-world use of FL in a cross-silo scenario with multiple companies. The companies aim to train a model collaboratively and use the resulting model in critical infrastructure. To enable collaboration, we introduced containers that allow the participants to include their input in the configuration of the FL process and decide on a suitable process. Furthermore, we introduced a container for managing the users and their devices for authentication in order to include only trusted clients in the process. In addition, the architecture is designed to store and track all decisions and information about the process. Besides the architecture, we analyzed the requirements of the companies and included them in our scenario-based evaluation. In the evaluation, we analyzed how the architecture accommodates the application's context and future changes. Following the evaluation results, we showed that the architecture design is suitable for our real-world application. We do not claim that the architecture is completed. It will be further improved to adjust to an extended range of scenarios and specific cases, including multiple real-world industrial field studies.

References

[1] F. Pallas, P. Raschke, and D. Bermbach, "Fog computing as privacy enabler," in *Internet Computing*. IEEE, 2020.

[2] B. McMahan, E. Moore, D. Ramage *et al.*, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proc. of the 20th Int. Conf. on Artificial Intelligence and Statistics*. PMLR, Apr. 2017, pp. 1273–1282.

[3] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data Poisoning Attacks Against Federated Learning Systems," Aug. 2020.

[4] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *2019 IEEE symposium on security and privacy (SP)*. IEEE, 2019, pp. 691–706.

[5] Q. Li, Y. Diao, Q. Chen, and B. He, "Federated learning on non-iid data silos: An experimental study," in *2022 IEEE 38th Int. Conf. on data engineering (ICDE)*. IEEE, 2022, pp. 965–978.

[6] A. Hard, K. Rao, R. Mathews *et al.*, "Federated Learning for Mobile Keyboard Prediction," Feb. 2019.

[7] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. on Intelligent Systems and Technology (TIST)*, vol. 10, no. 2, pp. 1–19, 2019.

[8] K. Pillutla, S. M. Kakade, and Z. Harchaoui, "Robust aggregation for federated learning," *IEEE Trans. on Signal Processing*, vol. 70, pp. 1142–1154, 2022.

[9] S. K. Lo, Q. Lu, H.-Y. Paik, and L. Zhu, "FLRA: A Reference Architecture for Federated Learning Systems," Jun. 2021.

[10] K. Bonawitz, H. Eichner, W. Grieskamp *et al.*, "Towards Federated Learning at Scale: System Design," Mar. 2019.

[11] B. Stojkovic, J. Woodbridge, Z. Fang *et al.*, "Applied federated learning: Architectural design for robust and efficient learning in privacy aware settings," 2022.

[12] Q. Cheng and G. Long, "Federated Learning Operations (FLOps): Challenges, Lifecycle and Approaches," in *2022 Int. Conf. on Technologies and Applications of Artificial Intelligence (TAAI)*. Tainan, Taiwan: IEEE, Dec. 2022, pp. 12–17.

[13] H. Ludwig, N. Baracaldo, G. Thomas *et al.*, "IBM Federated Learning: An Enterprise Framework White Paper V0.1," Jul. 2020.

[14] H. R. Roth, Y. Cheng, Y. Wen *et al.*, "NVIDIA FLARE: Federated Learning from Simulation to Real-World," 2022.

[15] D. J. Beutel, T. Topal, A. Mathur *et al.*, "Flower: A friendly federated learning research framework," *arXiv preprint arXiv:2007.14390*, 2020.

[16] B. A. Galende, S. U. Mayoral, F. M. García, and S. B. Lottmann, "Flip: A new approach for easing the use of federated learning," *Applied Sciences*, vol. 13, no. 6, p. 3446, 2023.

[17] M. N. Galtier and C. Marini, "Substra: a framework for privacy-preserving, traceable and collaborative machine learning," 2019.

[18] J. A. Peregrina, G. Ortiz, and C. Zirpins, "Towards Data Governance for Federated Machine Learning," in *Advances in Service-Oriented and Cloud Computing*, ser. Communications in Computer and Information Science. Cham: Springer Nature Switzerland, 2022, pp. 59–71.

[19] ——, "Towards a Metadata Management System for Provenance, Reproducibility and Accountability in Federated Machine Learning," in *Advances in Service-Oriented and Cloud Computing*, ser. Communications in Computer and Information Science. Cham: Springer Nature Switzerland, 2022, pp. 5–18.

[20] A. Z. Tan, H. Yu, L. Cui, and Q. Yang, "Towards personalized federated learning," *IEEE Trans. on Neural Networks and Learning Systems*, 2022.

[21] Y. Fraboni, R. Vidal, and M. Lorenzi, "Free-rider attacks on model aggregation in federated learning," in *Int. Conf. on Artificial Intelligence and Statistics*. PMLR, 2021, pp. 1846–1854.

[22] J. Zhang, J. Chen, D. Wu *et al.*, "Poisoning attack in federated learning using generative adversarial nets," in *2019 18th IEEE Int. Conf. on trust, security and privacy in computing and communications/13th IEEE Int. Conf. on big data science and engineering (TrustCom/BigDataSE)*. IEEE, 2019, pp. 374–380.

[23] E. Bagdasaryan, A. Veit, Y. Hua *et al.*, "How to backdoor federated learning," in *Int. Conf. on artificial intelligence and statistics*. PMLR, 2020, pp. 2938–2948.

[24] R. Kapitza, J. Behl, C. Cachin *et al.*, "Cheapbft: Resource-efficient byzantine fault tolerance," in *Proc. of the 7th ACM european Conf. on Computer Systems*, 2012, pp. 295–308.

[25] J. Park and H. Lim, "Privacy-preserving federated learning using homomorphic encryption," *Applied Sciences*, vol. 12, no. 2, p. 734, 2022.

[26] R. Kazman, G. Abowd, L. Bass, and P. Clements, "Scenario-based analysis of software architecture," *IEEE Software*, vol. 13, no. 6, pp. 47–55, Nov./1996.

Authorized licensed use limited to: INTERNATIONAL INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 07,2026 at 19:05:03 UTC from IEEE Xplore. Restrictions apply.